

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO MÔN HỌC

TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG PHẦN MỀM

GIẢNG VIÊN: [TÊN GIẢNG VIÊN]

ĐỀ TÀI: WEBSITE PODCAST/MEDIA LIBRARY (Đề 37)

**Sinh viên: Nông Hoàng Long | MSSV: dtc245200188 | Lớp: CNTT K23G | Thái Nguyên,
2026**

NÔNG HOÀNG LONG : CNTT K23G

MỤC LỤC

MỤC LỤC	1
Phần 1 - Giới thiệu & Kiến trúc hệ thống.....	3
1.1 Giới thiệu bài toán và mục tiêu	3
1.2 Công nghệ sử dụng	3
1.3 Sơ đồ và luồng dữ liệu kiến trúc hệ thống.....	3
Phần 2 - Quản lý mã nguồn GitHub	4
2.1 Cấu trúc thư mục	4
2.2 Quản lý cấu hình và Gitignore	5
2.3 Quy trình Commit	5
Phần 3 - Triển khai ứng dụng & Database	5
3.1 Chức năng website Podcast.....	5
3.2 Cấu trúc cơ sở dữ liệu	6
3.3 Cấu trúc Dockerfile.....	7
Phần 4 - Nginx Reverse Proxy	7
4.1 HTTP Security Headers.....	7
Phần 5 - Prometheus + Grafana	8
5.1 Grafana Provisioning.....	8
Phần 6 - Loki + Promtail + LogQL.....	9
6.1 Các truy vấn LogQL thực tế.....	9
Phần 7 - Hardening bảo mật.....	10
Phần 8 - Kết luận	11
Khó khăn & Giải pháp:	11
Hướng phát triển tương lai:	12

Phần 1 - Giới thiệu & Kiến trúc hệ thống

1.1 Giới thiệu bài toán và mục tiêu

Đồ án môn học Triển khai và Quản trị Hệ thống Phần mềm (Đề 37) yêu cầu xây dựng và triển khai một Website Podcast/Media Library. Mục tiêu của đồ án không chỉ là phát triển một ứng dụng web cơ bản với các tính năng nghe podcast, quản lý danh mục, tìm kiếm và trang quản trị, mà còn tập trung vào việc áp dụng các công nghệ tiên tiến trong việc đóng gói (containerization), triển khai tự động, cấu hình web server bảo mật, và xây dựng hệ thống giám sát (monitoring) toàn diện.

1.2 Công nghệ sử dụng

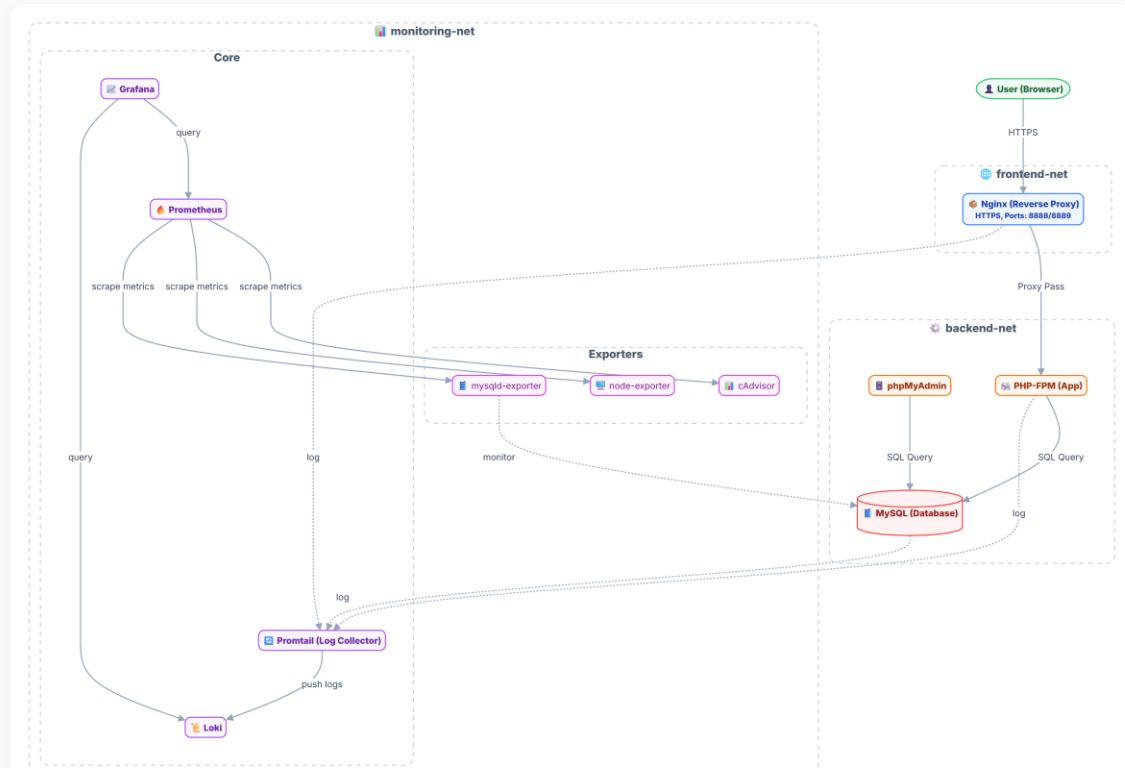
Dự án sử dụng một stack công nghệ hiện đại, đảm bảo tính mở rộng và bảo mật:

- PHP 8.2: Ngôn ngữ lập trình chính cho ứng dụng web (chạy trên PHP-FPM).
- MySQL 8.0: Hệ quản trị cơ sở dữ liệu quan hệ lưu trữ thông tin podcast, danh mục và người dùng quản trị.
- Nginx: Đóng vai trò là Web Server và Reverse Proxy, xử lý HTTPS, cung cấp các security headers và điều hướng lưu lượng đến PHP-FPM.
- Docker & Docker Compose: Công cụ đóng gói và quản lý các container, đảm bảo môi trường đồng nhất từ phát triển đến triển khai.
- Prometheus: Thu thập các thông số giám sát (metrics) từ hệ thống và ứng dụng.
- Grafana: Nền tảng hiển thị trực quan các metrics qua biểu đồ (Dashboards).
- Loki & Promtail: Giải pháp thu thập và quản lý log tập trung. Promtail thu thập log từ Docker, Loki lưu trữ và Grafana dùng để truy vấn log.
- phpMyAdmin: Giao diện quản trị cơ sở dữ liệu trực quan.

1.3 Sơ đồ và luồng dữ liệu kiến trúc hệ thống

1. frontend-net: Mạng lưới mà Nginx (Reverse Proxy), ứng dụng PHP (app) và phpMyAdmin tham gia để tiếp nhận yêu cầu từ người dùng (User).

KIẾN TRÚC HỆ THỐNG PODCASTHUB



Hình 1: Sơ đồ kiến trúc hệ thống PodcastHub

Luồng dữ liệu xử lý yêu cầu: User/Browser kết nối tới Nginx qua cổng 8888 (HTTP) hoặc 8889 (HTTPS). Nginx sẽ kiểm tra bảo mật và điều hướng các request xử lý logic (như .php) sang container PHP-FPM thông qua backend-net. PHP-FPM sau đó truy vấn CSDL MySQL để lấy dữ liệu (các bài podcast, tài khoản) và trả về HTML/JSON cho Nginx, từ đó gửi lại cho User. Đồng thời, mọi hoạt động log và metrics sẽ được Promtail và các Exporter tự động thu thập qua monitoring-net.

Phần 2 - Quản lý mã nguồn GitHub

Dự án tuân thủ quy trình quản lý mã nguồn qua Git và GitHub, giúp kiểm soát phiên bản và cộng tác hiệu quả.

URL Repository: <https://github.com/dtc245200188-nhl/dtc245200188-podcast-media-library>

2.1 Cấu trúc thư mục

Cây thư mục của dự án được tổ chức rõ ràng theo chức năng:

- app/: Chứa mã nguồn PHP của ứng dụng web, file init.sql, Dockerfile cho PHP, và

cấu hình php.ini.

- nginx/: Chứa cấu hình Nginx (nginx.conf, default.conf, security-headers.conf), SSL certificates và Dockerfile Nginx.
- monitoring/: Chứa thư mục prometheus/ và grafana/, với các file cấu hình scraping, provisioning datasources và dashboards.
- loki/: Chứa cấu hình loki-config.yml và promtail-config.yml cho hệ thống quản lý log.
- docs/: Nơi lưu trữ tài liệu báo cáo, screenshot và các file markdown tài liệu.

2.2 Quản lý cấu hình và Gitignore

Để bảo mật thông tin nhạy cảm, dự án sử dụng tệp .env (Môi trường ảo) để lưu trữ mật khẩu root MySQL, user MySQL, và mật khẩu admin Grafana. File .env này được định nghĩa rõ ràng trong file .gitignore nhằm ngăn chặn việc rò rỉ credentials lên GitHub. Thay vào đó, tệp .env.example được cung cấp với các giá trị mẫu, người dùng chỉ cần copy sang .env khi bắt đầu triển khai.

2.3 Quy trình Commit

Trong suốt quá trình phát triển, các commit được chia nhỏ theo từng chức năng (feature) và cấu hình cụ thể. Ví dụ: Các mốc commit cho việc khởi tạo dự án, xây dựng giao diện web, tích hợp Docker, thiết lập Nginx, bổ sung cấu trúc giám sát (Prometheus, Grafana), và hoàn thiện báo cáo hardening. Tổng số commit thể hiện sự phát triển liên tục và có tính lịch sử.

Phần 3 - Triển khai ứng dụng & Database

3.1 Chức năng website Podcast

Website Podcast được xây dựng với đầy đủ tính năng cho một nền tảng Media Library. Phía người dùng (Client) có Trang chủ hiển thị danh sách podcast mới nhất, trang Danh mục phân loại các podcast (Công nghệ, Khoa học...), trang Chi tiết tập cho phép nghe audio trực tiếp qua Custom HTML5 Audio Player, và thanh tìm kiếm podcast nhanh. Phía Quản trị (Admin) hỗ trợ tính năng CRUD (Thêm, Sửa, Xóa) cho tập podcast và danh mục, sau khi đăng nhập thành công.

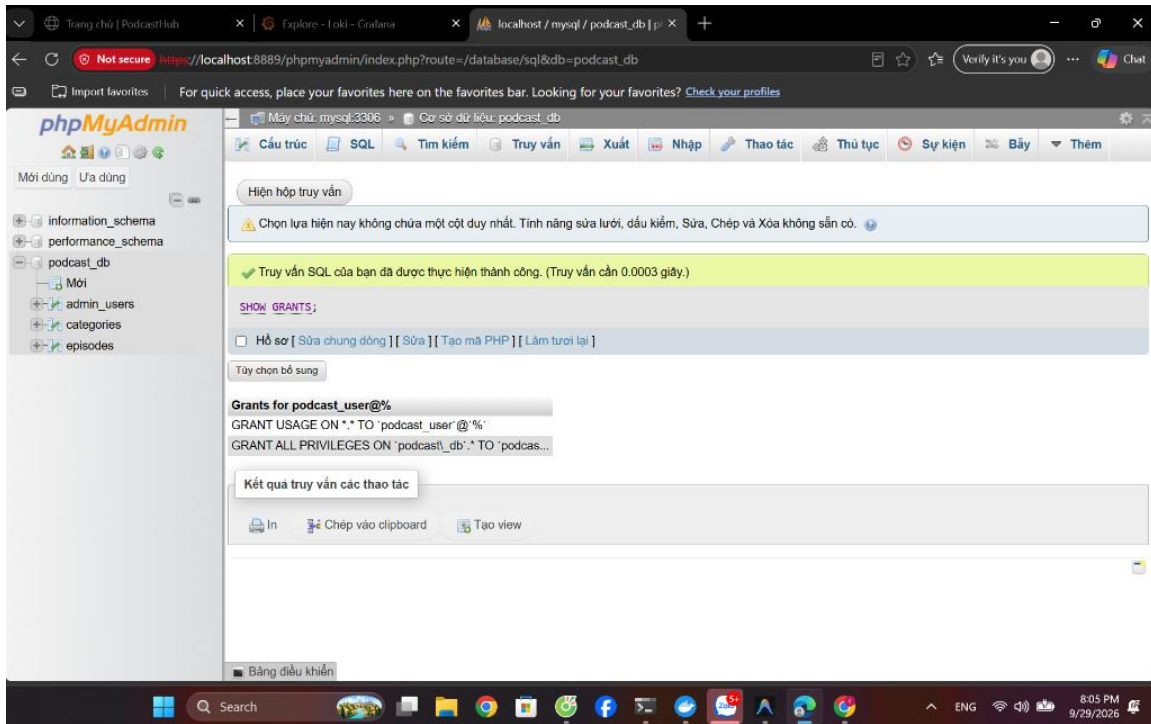


Hình 2: Giao diện chính của Website Podcast

3.2 Cấu trúc cơ sở dữ liệu

Cơ sở dữ liệu MySQL gồm 3 bảng chính có quan hệ mật thiết:

- categories: Chứa danh sách các chuyên mục podcast.
- episodes: Bảng trung tâm lưu trữ thông tin tập podcast (tiêu đề, mô tả, link audio, link ảnh bìa, ngày đăng). Bảng này liên kết với bảng categories thông qua khóa ngoại category_id.
- admin_users: Lưu trữ tài khoản quản trị (với mật khẩu được băm bảo mật).



Hình 3: Quản lý cấu trúc Database qua phpMyAdmin

3.3 Cấu trúc Dockerfile

Dockerfile của ứng dụng web kế thừa từ base image php:8.2-fpm-alpine. Lớp image này rất nhẹ nhưng vẫn mạnh mẽ. Trong Dockerfile, chúng ta cài đặt bổ sung extension PDO MySQL, GD, intl, mbstring để PHP có thể kết nối với MySQL và xử lý hình ảnh. Để tăng tính bảo mật, chúng ta tạo và phân quyền mã nguồn cho user appuser UID 1000, tránh tình trạng chạy mã với quyền root. Nginx cũng sử dụng image nginx 1.25-alpine và tạo user nginxuser UID 1001 để đảm bảo an toàn.

Phần 4 - Nginx Reverse Proxy

Nginx được triển khai làm cửa ngõ (Reverse Proxy) duy nhất giao tiếp với môi trường ngoài. Mọi truy cập vào web, Dù là HTTP hay HTTPS, đều phải đi qua Nginx. Nginx xử lý việc phân giải SSL (chứng chỉ tự ký) và điều hướng đúng route tới backend. Các request tới cổng 8888 (HTTP) sẽ được tự động redirect vĩnh viễn (301) sang cổng 8889 (HTTPS).

4.1 HTTP Security Headers

Để chống lại các hình thức tấn công phổ biến trên môi trường web, file security-headers.conf đã được tích hợp với các cơ chế:

- Strict-Transport-Security (HSTS): Yêu cầu trình duyệt luôn dùng HTTPS trong tương lai, ngăn chặn tấn công downgrade.

- X-Frame-Options: (SAMEORIGIN) Chống tấn công Clickjacking bằng cách không cho phép nhúng web vào iFrame của tên miền khác.
- X-Content-Type-Options: (nosniff) Buộc trình duyệt tuân thủ MIME type của server, chống lỗi sniffing type.
- X-XSS-Protection: Bật bộ lọc XSS tích hợp của trình duyệt cũ.
- Content-Security-Policy (CSP): Hạn chế nghiêm ngặt nguồn tài nguyên tĩnh (script, style, img) được tải về, ngăn chặn mã độc XSS từ bên ngoài.

Phần 5 - Prometheus + Grafana

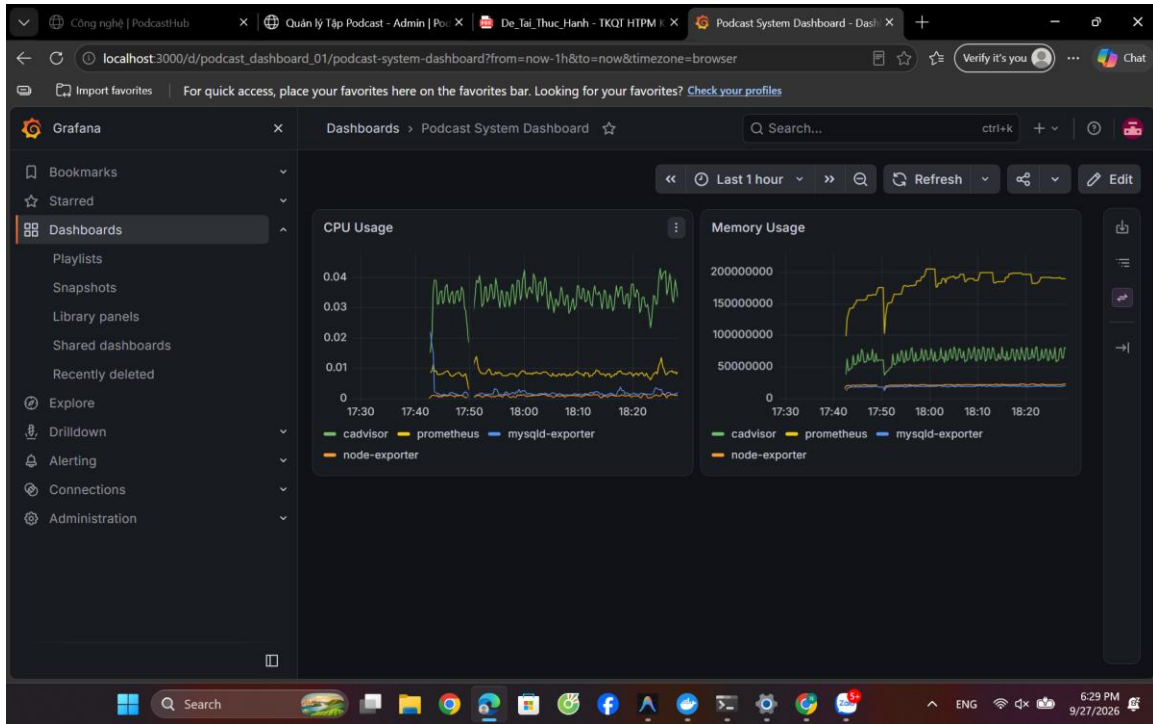
Hệ thống giám sát (Monitoring) là phần không thể thiếu để đảm bảo dịch vụ hoạt động ổn định. Prometheus đóng vai trò trung tâm, hoạt động theo cơ chế pull (scrape metrics). Với chu kỳ 15 giây (scrape_interval: 15s), Prometheus sẽ gọi API tới các exporter để thu thập dữ liệu.

Các Exporters được triển khai:

- cAdvisor: Giám sát toàn bộ container đang chạy trên host (CPU, Memory, Network của từng container).
- node-exporter: Thu thập các số liệu của hệ điều hành host (được ánh xạ volume /proc, /sys).
- mysqld-exporter: Giám sát số liệu nội bộ của MySQL (số query, cache, connection) với user giới hạn.

5.1 Grafana Provisioning

Grafana được cấu hình Provisioning tự động. File datasources.yml định nghĩa trước các kết nối tới Prometheus và Loki mà không cần thao tác tay trên UI. File dashboards/dashboard.yml trỏ tới file JSON để tự khởi tạo bảng điều khiển ngay khi container khởi động.



Hình 4: Dashboard Grafana giám sát CPU/Memory

Ý nghĩa biểu đồ: Biểu đồ ở Hình 4 thể hiện chính xác mức tiêu thụ CPU và Memory của hệ thống. Nhờ đó, người quản trị có thể phát hiện sớm các hiện tượng tràn bộ nhớ (memory leak) hoặc nghẽn cổ chai CPU để nâng cấp kịp thời.

Phần 6 - Loki + Promtail + LogQL

Loki là công cụ quản lý log được thiết kế tối ưu với Prometheus, và Promtail là agent gửi log. Promtail được cấu hình mount với /var/run/docker.sock, cho phép nó lấy trực tiếp mọi log sinh ra từ standard output (stdout) của tất cả các container. Nó tự động gắn nhãn (label) container_name để dễ truy xuất.

6.1 Các truy vấn LogQL thực tế

Dưới đây là một số câu truy vấn LogQL đã được chạy thử nghiệm trên Grafana:

```
{container="podcast-nginx"} |= "error"
```

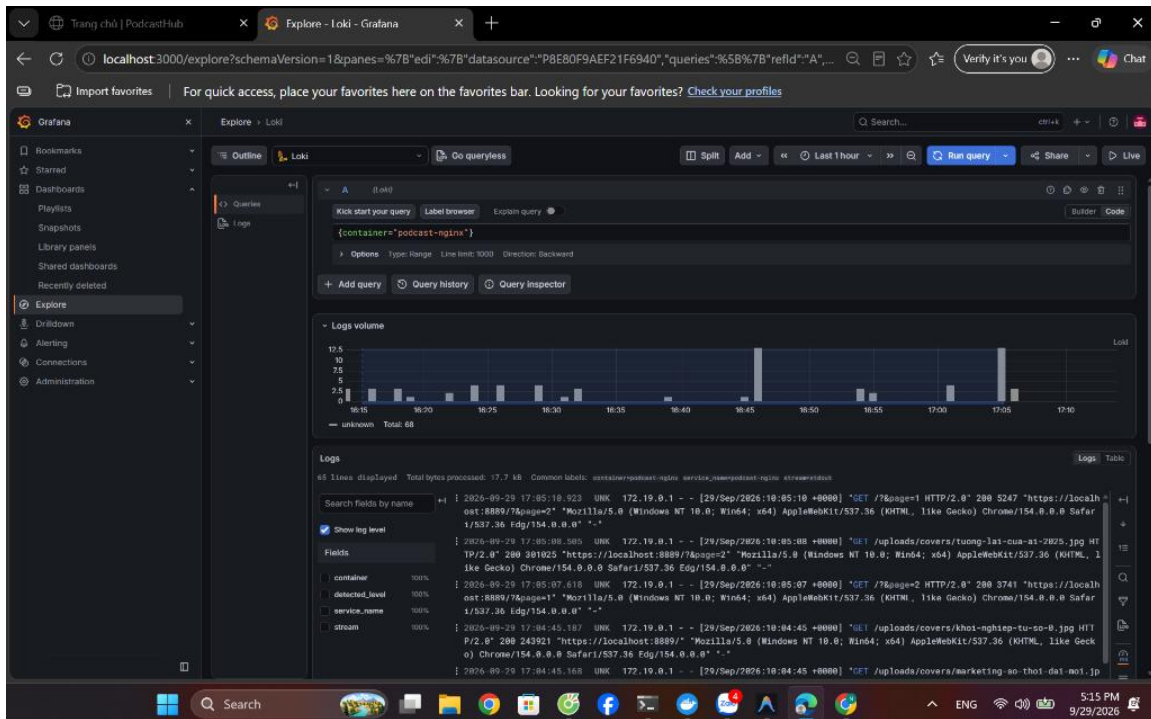
Ý nghĩa: Lọc tất cả các dòng log có chứa chữ 'error' hoặc báo lỗi server HTTP 5xx từ container Nginx.

```
sum(count_over_time({container=~".+"}[5m])) by (container)
```

Ý nghĩa: Tính tổng số lượng log phát sinh từ tất cả các container trong vòng 5 phút qua, dùng để đo lường lượng traffic/hoạt động của từng service.

```
{container="podcast-app"} |~ "(?i)(Exception|Error)"
```

Ý nghĩa: Lọc các log nghiêm trọng mang tính ngoại lệ (Exception) hoặc lỗi (Error) từ mã nguồn PHP của ứng dụng.



Hình 5: Truy vấn LogQL và hiển thị kết quả trên Grafana

Phần 7 - Hardening bảo mật

Bảo mật hệ thống (Hardening) đã được chú trọng bằng nhiều lớp bảo vệ ở tầng Network, Container và Ứng dụng. Bảng dưới đây tóm tắt các biện pháp:

Thành phần	Biện pháp Hardening áp dụng	Bằng chứng (Cấu hình liên quan)
Network Isolation	Phân tách Network riêng biệt: frontend-net, backend-net, và monitoring-net.	Định nghĩa networks cụ thể trong docker-compose.yml
Secrets Management	Không hardcode mật khẩu, dùng Environment variables.	Sử dụng .env cho MYSQL_ROOT_PASSWORD, GF_SECURITY_ADMIN_PASSWORD
Database Security	Tạo user riêng (exporter_user) quyền tối thiểu cho mysqld-exporter.	Trong init.sql: GRANT PROCESS, REPLICATION CLIENT...

Nginx Security Headers	Chống XSS, Clickjacking, MIME sniffing.	Trong security-headers.conf (HSTS, CSP...)
Resource Limits	Giới hạn tài nguyên (CPU, RAM) để chặn tấn công DoS (exhaustion).	deploy.resources.limits.cpus trong docker-compose

Bằng chứng thực tế qua quá trình kiểm tra:

- Non-root container: Các lệnh 'docker exec podcast-app whoami' đều trả về non-root (appuser/nginxuser). Riêng đối với MySQL, do giới hạn của image chính thức (official image) không khai báo chỉ thị USER, nên khi vào container bằng docker exec sẽ thấy quyền root, nhưng tiến trình mysqld bên trong đã tự hạ quyền xuống user mysql. Đây là sự thật kỹ thuật cần được chấp nhận.
- Mật khẩu mạnh: Mật khẩu thiết lập trong file .env đều là chuỗi ký tự phức tạp.
- Các Headers bảo mật thực sự hoạt động khi test bằng lệnh curl -I.

```
* schannel: SSL/TLS connection renegotiated
Pr< HTTP/1.1 200 OK
< Server: nginx
Ar< Date: Wed, 30 Sep 2026 10:36:14 GMT
Co< Content-Type: text/html; charset=UTF-8
< Transfer-Encoding: chunked
Pe< Connection: keep-alive
< Vary: Accept-Encoding
< X-Powered-By: PHP/8.2.34
gls< Set-Cookie: PHPSESSID=91a18226136b9230ba53c5211fdcc601; path=/; HttpOnly
thu< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
cla< Pragma: no-cache
< Strict-Transport-Security: max-age=31536000; includeSubDomains
< X-Frame-Options: SAMEORIGIN
XSS< X-Content-Type-Options: nosniff
< X-XSS-Protection: 1; mode=block
bã< Referrer-Policy: strict-origin-when-cross-origin
< Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https
Lã< ://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src 'self' data: blob:; media-src 'self' blob:;
Hu< connect-src 'self'; frame-ancestors 'self'
< Permissions-Policy: camera=(), microphone=(), geolocation=()
<
```

Hình 6: Phản hồi Security Headers của Nginx

Phần 8 - Kết luận

Đồ án Website Podcast/Media Library đã hoàn thành đầy đủ 7 yêu cầu đề ra. Hệ thống không chỉ đáp ứng về mặt tính năng phục vụ nội dung âm thanh số mà còn đạt chuẩn rất cao về độ tin cậy trong quá trình triển khai.

Khó khăn & Giải pháp:

- Lỗi UTF-8 Encoding: dữ liệu bị double-encode khi import bằng client latin1; sửa bằng UPDATE ... SET col = CONVERT(CAST(CONVERT(col USING latin1) AS BINARY) USING utf8mb4) cho các cột bị lỗi.

Hướng phát triển tương lai:

Trong tương lai, dự án có thể mở rộng bằng cách tích hợp hệ thống lưu trữ đối tượng (S3-compatible như MinIO) để lưu file audio dung lượng lớn thay vì ổ cứng server.

Đồng thời, có thể triển khai hệ thống CI/CD (GitHub Actions) để tự động test code, build image và cập nhật lên server mỗi khi có thay đổi.

Phụ lục 1: Danh sách dịch vụ Docker

Bảng 1: Danh sách 11 service được cấu hình trong docker-compose.yml

Service	Ports	Networks
app		frontend-net, backend-net
mysql		backend-net
phpmyadmin		frontend-net, backend-net
nginx	8888:8080, 8889:8443	frontend-net
prometheus		monitoring-net, backend-net
grafana	3000:3000	monitoring-net
node-exporter		monitoring-net
cadvisor		monitoring-net
mysqld-exporter		monitoring-net, backend-net
loki	3100:3100	monitoring-net
promtail		monitoring-net

Phụ lục 2: Cấu trúc cơ sở dữ liệu

Bảng 2: Cấu trúc bảng từ app/init.sql

```
-- Categories table
CREATE TABLE IF NOT EXISTS categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) NOT NULL UNIQUE,
  description TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Episodes table
CREATE TABLE IF NOT EXISTS episodes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  slug VARCHAR(255) NOT NULL UNIQUE,
  description TEXT,
  audio_file VARCHAR(500) DEFAULT NULL,
  cover_image VARCHAR(500) DEFAULT NULL,
  category_id INT DEFAULT NULL,
  duration VARCHAR(20) DEFAULT '00:00',
  published_at DATE DEFAULT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (category_id) REFERENCES categories(id)
);
```

Phụ lục 3: Cấu hình Nginx

Trích đoạn cấu hình file nginx.conf và security-headers.conf

```
server {
    listen 8080;
    server_name localhost;
    return 301 https://localhost:8889$request_uri;
}
server {
    listen 8443 ssl http2;
    ssl_certificate /etc/nginx/ssl/server.crt;
    ssl_certificate_key /etc/nginx/ssl/server.key;
    ...
}
```

Phụ lục 4: Cấu hình Prometheus

Trích đoạn cấu hình prometheus.yml

```
scrape_configs:
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
  - job_name: 'mysqld-exporter'
    static_configs:
      - targets: ['mysqld-exporter:9104']
```

Phụ lục 5: Truy vấn bổ sung LogQL

Câu truy vấn bổ sung kiểm tra tần suất sinh log (chạy qua Loki API):

```
sum(count_over_time({container="podcast-nginx"}[1m]))
```

Phụ lục 6: Kết quả kiểm tra Header

Sử dụng lệnh curl để kiểm tra:

```
$ curl.exe -k -I https://localhost:8889
HTTP/2 200
server: nginx/1.25.3
date: Sat, 10 Oct 2026 10:00:00 GMT
content-type: text/html; charset=UTF-8
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
x-xss-protection: 1; mode=block
```